**ALTOTECH GLOBAL CO., LTD.**

**Full-Stack Developer (Backend/Infra Focus)**

**Technical Assessment**

Multi-Site AFDD Service with Brick Schema Integration

**Graph Database Edition (v2)**

***"Detect. Diagnose. Optimize."***

# 1. Assessment Overview

This assessment evaluates your ability to build scalable cloud services for multi-site building automation. You will create a Multi-Site AFDD (Automated Fault Detection & Diagnostics) Service that helps AltoTech scale from 60+ to 400 properties by providing intelligent fault detection with Brick Schema integration, modeled natively on a graph database.

## Key Objectives

- **Master Building Management Systems (BMS):** Gain hands-on experience with real-world building automation concepts, equipment hierarchies, and sensor data flows that power smart buildings globally.
- **Model Buildings as Graphs with Brick Schema:** Buildings are inherently graphs of equipment, points, and locations, and Brick Schema is itself a graph ontology (RDF). Learn and implement this industry-standard model natively in a graph database so topology and semantics are first-class, not bolted-on tags.
- **Demonstrate Design-First Engineering:** Showcase your ability to architect systems before coding — producing clear documentation, thoughtful technical decisions, and scalable designs.
- **Deliver Production-Quality Code:** Build a working proof-of-concept that demonstrates clean architecture, proper testing, and deployment readiness.

## Goal

Design and build an AFDD platform that enables proactive building maintenance. The platform should answer: "What equipment needs attention, and why?"

## Success Criteria

| Outcome | Metric |
|---|---|
| Multi-site scalability | 3 hotels supported with isolated configs |
| Semantic data modeling | Brick Schema modeled natively as a graph (equipment/sensors as typed nodes) |
| Fault detection accuracy | 3-4 configurable AFDD rules implemented |
| Design-first approach | Architecture docs before implementation |

## 2. Problem Context

As AltoTech scales to 400 properties, we need unified fault detection that:

- Works across multiple sites with different configurations
- Uses semantic modeling (Brick Schema) for scalable equipment mapping
- Integrates with analytics dashboards, digital twins, and third-party APIs
- Is designed for future AI-driven rule discovery and cross-site learning

### Key Principle

***"Buildings are graphs. Brick Schema is a graph. Model the building as a graph — not as rows with sensor IDs."***

A graph database lets equipment, points, and locations be typed nodes connected by Brick relationships (hasPoint, isPointOf, feeds, hasLocation). "Find all AHUs on Floor 2" or "which sensors belong to Room 101" become natural graph traversals rather than multi-table joins.

### Building Management Systems (BMS) Background

Modern buildings contain interconnected systems that require intelligent monitoring:

- HVAC Systems: AHUs, VAVs, chillers, cooling towers — control temperature and air quality
- Electrical Systems: Power meters, lighting controls, plug loads — monitor energy consumption
- Sensors: Temperature, humidity, CO2, occupancy — provide real-time environmental data
- BMS (Building Management System): Central control integrating all subsystems

## 3. What We're Evaluating

| Area | Focus |
|---|---|
| Technical Execution | Can you deliver working, production-quality code? |
| Brick Schema | Can you model building semantics natively as a graph? |
| System Design | Can you design scalable, multi-site services? |
| Design Thinking | Do you think and document before coding? |
| Integration Thinking | Do you consider external systems and future AI capabilities? |

### Deliverable Expectations

| Type | Expectation |
|---|---|
| Working Code | Primary focus — functional proof-of-concept with clean architecture, proper error handling, and test coverage |
| Design Documentation | Architecture diagrams, ADRs, and technical specs that demonstrate thoughtful planning before implementation |
| Scalability Thinking | Evidence that you've considered growth to 100+ sites and integration with external systems |

## 4. Assessment Tasks

Complete all four tasks below. Use AI-assisted tools to accelerate development.

### Task Overview

| Task | Focus | Weight |
|---|---|---|
| 1 | Foundation: Edge Simulation + Cloud Infra | 20% |
| 2 | Brick Schema Graph Modeling & Integration | 30% |
| 3 | Multi-Site AFDD Engine (Brick-aware, AI-ready design) | 35% |
| 4 | Deployment & Documentation | 15% |

### System Architecture Diagram and System Data Flow

Before diving into tasks, understand how data flows through the system:

*Edge Simulator → Ingestion API (+ Brick tagging via graph lookup) → Data Store → AFDD Engine → Fault Records → Alerts/Dashboard*

- Edge simulators push raw sensor data from hotel rooms
- Ingestion API receives data, resolves each device against the Brick graph, stores readings
- AFDD Engine periodically evaluates rules against recent data
- When faults detected, system creates fault records and can trigger notifications

**Note on storage:** Graph databases model topology and semantics extremely well but are not optimized for high-volume time-series writes. Part of this assessment is deciding where high-frequency sensor readings live versus what belongs in the graph. See Task 1.

### Task 1: Foundation — Edge Simulation + Cloud Infra (20%)

**Goal: Set up minimal edge simulation and cloud backend for multi-site data ingestion, with the building topology modeled in a graph database.**

**Scenario: 3 Hotels with Room Sensors**

Simulate 3 hotel properties, each with multiple guest rooms equipped with IoT sensors:

- Hotel A: 10 rooms, each with IAQ sensor (temp, humidity, CO2) + occupancy sensor
- Hotel B: 8 rooms, similar sensor setup + power meters per floor
- Hotel C: 12 rooms, full sensor deployment

Each sensor pushes data every 30-60 seconds. Include realistic fault patterns (stuck sensors, temperature spikes, schedule violations) for AFDD testing.

**Edge Simulator Requirements**

- Python script that simulates sensors for all 3 hotels
- Configurable: hotel count, rooms per hotel, sensor types, push interval
- Generates realistic data with embedded anomalies for testing
- Pushes to cloud ingestion API

**Graph Data Model Design**

Model the building as a graph of typed nodes and relationships. Sites, locations, and devices become nodes; their structure is expressed through Brick relationships. You choose the exact labels and properties — the reference below is a starting point, not a prescription.

| Node (label) | Purpose | Suggested Properties |
|---|---|---|
| Property | Hotel/site registry | id, name, timezone, config, created_at |
| Location | Rooms/zones within property | id, name, floor, type |
| Device / Point | Sensor/equipment registry | id, device_type, brick_class, metadata |
| Rule | AFDD rule configuration | id, name, brick_class_target, conditions, thresholds, enabled |
| Fault | Detected fault event | id, severity, status, detected_at, context |

**Suggested Relationships (Brick-aligned):**

- (:Property)-[:hasPart]->(:Location)
- (:Location)-[:hasPoint]->(:Device)
- (:Device)-[:isPointOf]->(:Equipment)
- (:Rule)-[:targets]->(:BrickClass)
- (:Fault)-[:detectedOn]->(:Device)
- (:Fault)-[:raisedBy]->(:Rule)

**Time-series readings:** Each Device produces a stream of readings every 30-60s. Decide where this high-volume stream is stored and how it links back to the Device node in the graph. Document your decision and its tradeoffs in your design doc.

**Required API Functionalities**

Design and implement APIs that support these operations (you design the endpoints):

- Property & Device Setup: Basic CRUD for properties and devices as graph nodes (keep minimal — focus on what's needed for ingestion)
- Data Ingestion: Receive sensor data (batch and single), validate and resolve Brick metadata via graph lookup
- Data Query: Get latest readings, query time-range data by device

**Deliverables**

- edge_simulator/ with configurable script
- Working ingestion API with all required functionalities
- Graph schema / data model diagram (nodes + relationships) and load scripts
- Design doc explaining graph model decisions and where time-series data lives

## Task 2: Brick Schema Graph Modeling & Integration (30%)

**Goal: Model Brick Schema natively in your graph database so all devices are typed nodes connected by real Brick relationships, and ingestion resolves semantics through the graph.**

**What is Brick Schema?**

Brick is an open-source ontology (built on RDF, a graph data model) for describing building equipment, sensors, and their relationships. It provides a standardized vocabulary that enables:

- Portable applications across different buildings
- Equipment discovery by type (e.g., “find all AHUs”)
- Relationship queries (e.g., “which sensors are in Room 101?”)

Because Brick is already a graph, a graph database is its natural home: Brick classes become node types, and Brick relationships become edges.

**Critical: Brick Resolution at Ingestion**

**The key integration point is during data ingestion. When sensor data arrives:**

- Step 1: Receive raw sensor payload (device_id, datapoint, value, timestamp)
- Step 2: Look up the device node in the graph → read its Brick class (e.g., brick:Zone_Air_Temperature_Sensor) and its relationships (location, equipment)
- Step 3: Associate the reading with the resolved device node / Brick context
- Step 4: Store the reading in your chosen time-series store, keyed to the graph device

**Example Ingestion Flow:**

```
Incoming: {device_id: "sensor_101", datapoint: "temperature", value: 23.5}

Graph lookup (Cypher example):
  MATCH (d:Device {id: "sensor_101"})-[:hasLocation]->(l:Location)
  RETURN d.brick_class, l.name

Resolved: brick_class = "brick:Zone_Air_Temperature_Sensor",
          location   = "Room_101"
```

**Brick Class Mapping**

| Sensor Type | Brick Class | Description |
|---|---|---|
| Temperature | brick:Zone_Air_Temperature_Sensor | Measures room air temperature |
| Humidity | brick:Zone_Air_Humidity_Sensor | Measures room humidity level |
| CO2 | brick:CO2_Sensor | Measures carbon dioxide concentration |
| Occupancy | brick:Occupancy_Sensor | Detects room occupancy state |
| Power Meter | brick:Electrical_Power_Sensor | Measures electrical power consumption |

**Design Document Requirements**

- Explain Brick Schema concepts relevant to AFDD and why a graph model fits
- Document your Brick class hierarchy and the relationships you model as edges
- How AFDD rules target Brick classes via graph traversal (e.g., “all brick:Zone_Air_Temperature_Sensor”)
- How adding new Brick classes / nodes would auto-enable relevant AFDD rules

**External Integration Design**

- How this service would integrate with: Analytics dashboards, Digital Twin platforms, Third-party BMS
- API contracts for external consumers querying the graph by Brick class

**Deliverables**

- Brick-aware ingestion pipeline (graph resolution at ingest time)
- Device registry modeled as Brick-typed nodes with relationships
- APIs to query devices by Brick class via graph traversal (e.g., get all device_ids of brick:Temperature_Sensor on Floor 2)
- Brick Schema graph design document

## Task 3: Multi-Site AFDD Engine — Brick-Aware (35%)

**Goal: Build an automated fault detection engine that continuously monitors sensor data and identifies equipment issues.**

**What is an AFDD Engine?**

An AFDD (Automated Fault Detection & Diagnostics) engine is a background service that:

- Periodically evaluates sensor data against configured rules
- Detects anomalies, equipment faults, and operational issues
- Creates fault records with diagnostic context
- Can trigger alerts or notifications

**Engine Architecture**

**The AFDD engine consists of these components:**

- **Rule Scheduler:** Runs rule evaluations at configurable intervals (e.g., every 1-5 minutes)
- **Condition Evaluator:** Resolves target devices via the Brick graph, then checks fault conditions using time-windowed data queries
- **Fault Manager:** Creates, updates, and resolves fault records; prevents duplicate alerts
- **Alert Dispatcher:** Sends notifications when faults are detected (webhook, log, or queue)

**How Rule Evaluation Works**

Each rule evaluation follows this pattern:

- Traverse the Brick graph to resolve the set of target devices for the rule's Brick class (e.g., all brick:Zone_Air_Temperature_Sensor)
- Query recent readings for those devices from the time-series store (e.g., last 15 minutes)
- Apply condition logic (e.g., check if values exceed threshold for sustained period)
- If fault conditions met AND no active fault exists → create new fault
- If fault conditions cleared AND active fault exists → resolve the fault

**Example: Temperature Excursion Rule**

***"Alert if Zone_Air_Temperature_Sensor readings exceed 28°C for more than 10 consecutive minutes"***

- Target: brick:Zone_Air_Temperature_Sensor (resolved by graph traversal)
- Condition: value > 28 for duration > 10 minutes
- Window: Check last 15 minutes of data
- Output: Create fault with severity=warning, include sensor readings as context

**Required Fault Rules (Implement 3-4)**

| Rule | Detection Logic | Example Threshold |
|---|---|---|
| Temperature Excursion | Value outside setpoint ± threshold for X minutes | > 28°C for 10 min |
| Sensor Flatline | No variance (stddev ≈ 0) in readings for X minutes | 0 variance for 30 min |
| Schedule Violation | Equipment running outside defined operating hours | ON after 22:00 |

| Rule | Detection Logic | Example Threshold |
|---|---|---|
| Energy Anomaly | Consumption exceeds rolling average by X % | > 150% of 7-day avg |

**When Fault is Detected — Required Actions**

- Create fault record: links to rule + device (graph edges), severity, status='active', detected_at, context (recent readings)
- Prevent duplicates: Don't create new fault if active fault exists for same device+rule
- Log the event: Structured log with fault details for observability
- Dispatch alert: Call webhook or push to message queue (can be mock implementation)

**Fault Lifecycle**

detected → active → acknowledged → resolved

- detected: Fault conditions first met
- active: Fault ongoing, not yet addressed
- acknowledged: Operator has seen the fault
- resolved: Conditions no longer met OR manually closed

**Rule Configuration Schema**

**Rules should be configurable and stored as nodes in the graph. Below is a sample structure — design a format that suits your implementation best:**

```
{
  "name": "High Temperature Alert",
  "brick_class_target": "brick:Zone_Air_Temperature_Sensor",
  "condition_type": "threshold_exceeded",
  "threshold_value": 28,
  "duration_minutes": 10,
  "severity": "warning",
  "enabled": true,
  "property_overrides": { "hotel_a": { "threshold_value": 26 } }
}
```

**Required API Functionalities**

Design APIs supporting these operations:

- Rule Management: Create, read, update, delete rules; enable/disable rules per property
- Fault Queries: List active faults, filter by property/severity/status, get fault details with context
- Fault Actions: Acknowledge fault, resolve fault, add notes to fault
- Dashboard Data: Get fault counts by property, severity distribution, recent faults

**Scalability Design Doc**

- How would the engine scale to 100+ sites? (consider: job distribution, graph partitioning / sharding, query performance)
- How to add new rule types without code changes? (rule DSL, plugin architecture)

- How would cross-site patterns be detected? (aggregated graph analytics)

**AI-Ready Design Considerations**

Address in your design document:

- How would this engine export the graph / data / configs for AI analysis?
- What data format would enable AI tools (e.g., Claude) to suggest new rules?
- How could cross-site learnings be structured for AI consumption?

**Bonus: Claude Code Skill for Brick Schema & AFDD**

**Create a Claude Code skill (SKILL.md) that enables AI to understand and work with this system:**

- Brick Schema graph modeling patterns: How to map building equipment to Brick-typed nodes and relationships
- Common Brick queries: Cypher/SPARQL examples for querying sensors by type, location, relationships
- AFDD rule configuration: Schema and examples for creating valid rule configs
- Goal: Future AI can read this skill and generate valid AFDD rule configurations

Reference: https://docs.anthropic.com/en/docs/claude-code/skills

**Deliverables**

- Working AFDD engine with scheduler, evaluator, and fault manager
- 3-4 implemented fault rules
- Rule configuration API
- Fault management API
- Scalability and AI-ready design document
- (Bonus) Claude Code SKILL.md file

## Task 4: Deployment & Documentation (15%)

**Goal: Production-ready deployment setup with comprehensive documentation.**

**Required:**

- Docker Compose (API + Graph DB + time-series store + simulator + AFDD engine)
- Environment configs (dev/prod)
- Health check endpoints
- Structured logging
- OpenAPI/Swagger UI for API testing
- README for one-command startup

**Bonus:**

- Kubernetes manifests

- Basic monitoring (Prometheus metrics endpoint)
- Load test results

## 5. Deliverables

| # | Deliverable | Format |
|---|---|---|
| 1 | Working Prototype | Docker Compose deployment (one-command startup) |
| 2 | GitHub Repository | All code, tests, and documentation |
| 3 | GitHub Wiki | Technical documentation (architecture, diagrams, ADRs) |

### GitHub Wiki Requirements

- **System Architecture:** Visual representation of module connectivity, high-level component overview
- **Flow Charts & Sequence Diagrams:** Data flows for key operations, AFDD evaluation cycle
- **Architecture Decisions:** Key technical decisions with rationale (ADR-style), including graph DB choice and where time-series data lives
- **Brick Schema Design:** How the Brick ontology maps to your graph data model

## 6. Provided Assets

| Asset | Purpose |
|---|---|
| Edge simulator template | Reduce boilerplate work |
| Sample sensor data (CSV) | IAQ, occupancy, power meter data |
| Sample fault patterns | For AFDD testing |
| Brick Schema primer | Basic ontology reference |
| Evaluation rubric | Transparent expectations |

## 7. Mock Data Reference

Sensor reading format produced by the edge simulator. Note that brick_class is resolved at ingestion via the device's node in the graph rather than carried on the wire:

### Sensor Reading Format

| Field | Type | Description |
|---|---|---|
| timestamp | INTEGER | Unix timestamp |
| device_id | TEXT | Unique device identifier (maps to a Device node) |
| datapoint | TEXT | Measurement type (temperature, humidity, etc.) |
| value | TEXT | Measured value |
| brick_class | TEXT | Brick Schema class (resolved at ingestion from the graph) |

**Storage note:** High-frequency readings are best held in a time-series-capable store and linked to the Device node in the graph. Brick classes and relationships live in the graph; you decide the boundary and justify it.

## 8. Evaluation Criteria

| Criteria | Weight | What We Look For |
|---|---|---|
| AFDD Engine & Logic | 35% | Rule correctness, Brick-aware (graph-resolved) targeting, configurability, AI-ready considerations |
| Brick Schema Graph Modeling | 30% | Quality of the graph model, native Brick typing & relationships, resolution at ingestion |
| Cloud Infra & Data Design | 20% | Graph schema quality, multi-tenant design, sound graph-vs-time-series boundary, query efficiency |
| Deployment & Docs | 15% | Docker setup, documentation clarity |

### Cross-Cutting Evaluation

| Criteria | Evaluation |
|---|---|
| AI Tool Usage | Effective use of Claude Code, Cursor, or similar tools for development acceleration |
| GitHub Workflow | PR hygiene, meaningful commits, feature branches, code review readiness |
| Testing & Quality | Unit tests, integration tests, test coverage, CI pipeline setup |
| Design-first approach | Evidence of documentation before implementation |
| Scalability thinking | Consideration for 100+ sites, 1000+ sensors |
| Code quality | Clean, maintainable, well-structured code |

## 9. Recommended Tools

### AI-Assisted Development

- **Claude Code:** AI coding assistant for terminal — https://docs.anthropic.com/en/docs/claude-code
- **Cursor:** AI-powered code editor — https://cursor.com
- **GitHub Spec-Kit:** Spec-driven development with AI — https://github.com/github/spec-kit
- **TestSprite:** AI-powered test generation — https://www.testsprite.com

### Graph Database — Choose One

Pick the graph database you are most comfortable defending. Each has tradeoffs; document why you chose it:

| Option | Query Language | Tradeoff / When It Fits |
|---|---|---|
| Neo4j | Cypher | Most mature tooling and ecosystem; clean labelled-property graph. Needs a separate time-series store for high-volume readings. |
| Apache AGE | openCypher (on PostgreSQL) | Graph extension inside Postgres — hybrid graph + time-series (e.g., TimescaleDB) is natural in one engine. Younger ecosystem. |
| GraphDB / RDFLib (RDF) | SPARQL | Truest fit for Brick, which is RDF-native; direct ontology import. Steeper learning curve, weaker time-series story. |

### Development Stack

- FastAPI or Django — Backend framework
- Graph DB of your choice (see above) for topology + Brick
- Time-series-capable store (e.g., TimescaleDB) for high-volume readings, if your design separates them
- Docker Compose — Local deployment
- GitHub — Version control and documentation

### AltoTech Stack (Reference)

- Backend: Python - Django, FastAPI
- Database: SupabaseDB, TimescaleDB; graph DB for semantic/topology layer
- Cloud: Azure
- Deployment: Docker (scaling to Kubernetes)

## 10. Reference Resources

Use these resources to quickly understand the domain:

### Brick Schema

- Official Website: https://brickschema.org/
- Documentation: https://docs.brickschema.org/
- Ontology Explorer: https://ontology.brickschema.org/
- Brick Viewer (explore ontology): https://viewer.brickschema.org/
- Brick Schema GitHub: https://github.com/BrickSchema/Brick
- Medium Article (good intro): https://medium.com/@erik_paulson/the-brick-schema-data-portability-and-independent-data-layers-for-smart-buildings-2dc048efbf3b

### Graph Databases & Brick

- Neo4j Cypher Manual: https://neo4j.com/docs/cypher-manual/current/
- Apache AGE (Postgres graph extension): https://age.apache.org/
- GraphDB by Ontotext: https://graphdb.ontotext.com/
- RDFLib (Python RDF library): https://rdflib.readthedocs.io/
- Brick on a graph store (Brick + SPARQL): https://docs.brickschema.org/lite/querying.html

### Building Management Systems (BMS)

- CIBSE BMS Training (PDF): https://www.cibse.org/media/uhueio0f/bms-training-presentation.pdf
- BMS Beginner Guide: https://www.tmba.com/blog/guide-to-building-management-systems
- BMS Training Videos: https://www.bms-training.com/videos
- HVAC System 101: https://www.youtube.com/@EngineeringMindset
- Air Conditioning Basics: https://youtu.be/SfuSzBja8QA?si=5eBmyJxfrvD7KN1W

### Fault Detection & Diagnostics (FDD)

- Fault Detection and Diagnostics (FDD) 101: https://youtu.be/rvsTUiZBtsc?si=rqOWfN3Mmqn1uLoN
- What is FDD (Facilio): https://facilio.com/learn/fault-detection-and-diagnostics/
- Academic Review (PMC): https://pmc.ncbi.nlm.nih.gov/articles/PMC9824457/
- Real-World Case Study (52 buildings, 100K+ datapoints): https://facilities.uiowa.edu/fault-detection-and-diagnostics

### Claude Code & AI Development

- Claude Code Docs: https://docs.anthropic.com/en/docs/claude-code

- Creating Custom Skills: https://docs.anthropic.com/en/docs/claude-code/skills

## 11. Submission Instructions

- Push all code and documentation to your GitHub repository
- Complete the GitHub Wiki with architecture diagrams and design docs
- Ensure Docker Compose runs with a single command
- Include test results and coverage reports
- Be prepared for a real DEMO during the assessment review

## 12. Interview Follow-Up Questions

After submission, we will discuss:

- “Why did you choose a graph database here, and why your specific engine over the alternatives?”
- “Walk through your Brick Schema graph model. How does resolution work at ingestion?”
- “Where do high-volume time-series readings live, and how do they link back to the graph? What drove that boundary?”
- “Explain how your AFDD engine resolves rule targets via traversal and evaluates conditions. What happens when a fault is detected?”
- “How would you scale this to 100 sites? Where are the bottlenecks — graph queries, time-series, or job distribution?”
- “Show us how you used AI tools during development. What worked well?”
- “Walk us through your GitHub workflow and testing approach.”
- “What would you do differently with more time?”

**Good luck! We look forward to seeing your solution.**

*Remember, at AltoTech, collaboration is key. Consider us as your teammates and don't hesitate to reach out with questions.*